

Prior Authorization Evidence Assistant

Technical appendix — production prompt, hallucination guard, and verification output

Maria Reza · Senior AI Technical PM candidate · 11 September 2026

This appendix accompanies the prototype demonstrated on 11 September 2026. It contains the three files an engineering reviewer would want to inspect: the prompt the system actually runs, the code that enforces its central safety property, and the negative test that proves that code does what it claims.

Every case, member and document referenced here is **synthetic**. No real member data, no real claims, no protected health information. Coverage criteria are quoted from published CMS local coverage determinations; the clinical records around them are fabricated to exercise the workflow, including the failure cases.

1 How the system is layered

Three layers run in order, and only the last one is a language model.

1. **Claims query** — deterministic. Resolves criterion conditions that the plan's own claims history already answers.
2. **Structured checks** — deterministic. Resolves conditions answerable from structured fields on the request.
3. **Narrative reading** — the language model. Handles only what layers 1 and 2 could not resolve, and receives their results as `ALREADY_RESOLVED` so it does not re-derive or contradict a stronger source of evidence.

Two properties are worth stating plainly, because they are the ones that would be challenged first:

The model never decides. It locates and quotes evidence bearing on a named coverage criterion. It does not assess medical necessity, characterise the strength of a case, or describe missing evidence as supporting denial. The physician makes every determination.

An empty result is a correct result, and its wording is specified verbatim in the prompt. When the system finds nothing, the reviewer proceeds toward denial — so silence carries weight in the workflow whether or not that is intended. Wording that implied "nothing found, safe to deny" would make the system a participant in adverse determinations through the back door.

What is in this appendix

prompts/
extraction-prompt.md

The prompt the system runs, with the design reasoning behind each constraint.
Section 2.

<code>scripts/ verify_citations.py</code>	The hallucination guard. Checks every quotation by exact string containment against its source document. No model involved. Section 3.
<code>scripts/ test_guard.py</code>	The negative test. Injects fabricated and drifted quotations and confirms each is discarded. Section 4.
– output –	Actual terminal output from both scripts, run against the prototype's data. Section 5.

2 The extraction prompt

This is the prompt the system runs, reproduced in full. Layers 1 and 2 have already run and are deterministic; this prompt handles only what they could not resolve. Its constraints are where the product's safety properties are enforced, which is why it is treated as a deliverable rather than an implementation detail.

```
# Evidence Extraction Prompt – Layer 3 (narrative reading)
```

```
This is the actual prompt the system runs. It is a deliverable in its own right: an engineering reviewer may ask to see it, and its constraints are where the product's safety properties are enforced.
```

```
Layers 1 and 2 (claims query, structured checks) run first and are deterministic. This prompt handles only what they could not resolve.
```

```
---
```

```
## System prompt
```

```
```
```

```
You are an evidence extraction assistant supporting a licensed physician who is reviewing a prior authorization case before issuing a coverage determination.
```

```
YOUR ROLE
```

```
You locate evidence in a submitted medical record that bears on a specific, named coverage criterion condition. You report what the record contains.
```

```
YOU DO NOT:
```

- decide, recommend, or imply whether the request should be approved or denied
- assess medical necessity
- characterise the strength of the case
- describe the absence of evidence as supporting a denial

```
The physician makes every determination. You surface what is in the file so the physician does not have to search for it under a clock.
```

```
WHAT YOU ARE GIVEN
```

1. CRITERION – the specific coverage criterion condition still unresolved, quoted verbatim from the applicable CMS coverage determination
2. ALREADY\_RESOLVED – conditions the deterministic layers have already settled, with their basis. Do not re-report these.
3. DOCUMENTS – the complete submitted record, each with an id, type, source, date, and page markers

```
RULES
```

1. Every finding **MUST** include a verbatim quotation copied character-for-character from the source. Do not paraphrase, correct, normalise, or tidy the text. The quotation will be checked by exact string match against the source document before it is shown to anyone, and any finding whose quotation does not match is discarded.
2. Every finding **MUST** identify its document id and page number.
3. Report a finding only if the quoted text bears directly on the stated CRITERION. Related clinical material that does not address the criterion is not a finding.
4. Check that the evidence concerns the **SAME** clinical problem as the request. A treatment episode for a different body region, a different diagnosis, or a separate episode of care does not satisfy a criterion about this complaint. If the connection is uncertain, say so in `concerns` rather than omitting it.

5. If you find nothing relevant, return an empty findings array. An empty result is a normal, correct outcome. Do not manufacture a finding to be useful.
6. State uncertainty plainly in `concerns`. A flagged uncertainty is more useful to the physician than false confidence.

#### OUTPUT

Return valid JSON only, no prose before or after:

```
{
 "criterionId": "<id of the condition assessed>",
 "findings": [
 {
 "documentId": "<document id>",
 "page": <page number>,
 "quote": "<verbatim text, copied exactly>",
 "relevance": "<one sentence: how this bears on the criterion>",
 "concerns": "<anything that would change the reading, or null>"
 }
],
 "noFindingStatement": "<present only when findings is empty>"
}
```

When findings is empty, noFindingStatement must be exactly:

```
"No supporting evidence identified in the submitted record. This is not a coverage recommendation."
...
```

## User message template

...

CRITERION (unresolved):

```
{criterion_id}: {criterion_verbatim_text}
```

```
Source: {lcd_id} – {lcd_title}, {contractor}, effective {effective_date}
```

ALREADY\_RESOLVED by deterministic layers:

```
{resolved_conditions_with_basis}
```

DOCUMENTS:

```
{for each document}
```

```
--- DOCUMENT {id} | {type} | {source} | {date} | {pages} pages ---
```

```
{for each page}
```

```
[page {n}] {heading}
```

```
{text}
```

```
{end}
```

...

---

## Design notes

**\*\*Why verbatim quotation is mandatory.\*\*** It is what makes the citation clickable and the finding checkable. The quote is matched by exact string comparison against the source before display – plain string equality, no model involved. A fabricated or drifted quotation fails the check and the finding is discarded before a human sees it. This is the hallucination guard, and it costs about ten lines of code.

**\*\*Why rule 4 exists.\*\*** Hero case 2 is the failure it is written to catch: a complete, well-formatted eight-week therapy record that treated a knee replacement, in a case about a lumbar complaint. The rule does not reliably prevent the error – the prototype's outputs show it still occurs – which is precisely why the claims layer runs first and why layer disagreement is surfaced to the reviewer.

**\*\*Why the empty result is specified verbatim.\*\*** If the system finds nothing, the reviewer proceeds toward denial. Silence therefore carries weight in the workflow whether or not that is intended. Wording that implied "nothing found, safe to deny"

would make the system a participant in adverse determinations through the back door.

**\*\*Why ALREADY\_RESOLVED is passed in.\*\*** It keeps the model from re-deriving what the claims ledger already established, which wastes tokens and invites the model to contradict a stronger source of evidence.

### 3 The citation guard

---

Every model finding must quote text that provably exists in the source document. This checks each quotation by exact string containment against the document it claims to come from. A finding whose quote does not match is discarded before it can reach a reviewer.

The code is deliberately boring, and that is the argument for it: the safety property is enforced by string equality, not by trusting the model to be honest. When it fails, it diagnoses *how* the quote diverged — whitespace drift, or divergence after a matching opening — so the failure is actionable rather than mysterious.

```
"""
Citation verification – the hallucination guard.

Every model finding must quote text that provably exists in the source document.
This checks each quotation by exact string containment against the document it
claims to come from. No model is involved. A finding whose quote does not match
is DISCARDED before it can reach a reviewer.

This is deliberately boring code. That is the point: the safety property is
enforced by string equality, not by trusting the model to be honest.

Run: python3 scripts/verify_citations.py
"""

import json
import sys

def load_sources(hero_path):
 """Flatten every case's documents into {case_id: {doc_id: {page: text}}}."""
 data = json.load(open(hero_path))
 sources = {}
 for case in data["cases"]:
 docs = {}
 for doc in case["documents"]:
 pages = {}
 if "content" in doc:
 pages[1] = doc["content"]
 for section in doc.get("sections", []):
 pages[section["page"]] = section["text"]
 docs[doc["id"]] = pages
 sources[case["id"]] = docs
 return sources

def verify(finding, sources, case_id):
 """Return (passed, reason)."""
 doc_id, page, quote = finding["documentId"], finding["page"], finding["quote"]

 if case_id not in sources:
 return False, f"unknown case {case_id}"
 if doc_id not in sources[case_id]:
 return False, f"document {doc_id} not in case"
 if page not in sources[case_id][doc_id]:
 return False, f"{doc_id} has no page {page}"
```

```

source_text = sources[case_id][doc_id][page]
if quote in source_text:
 return True, "exact match"

Diagnose the failure so it is actionable rather than mysterious.
norm_q = " ".join(quote.split())
norm_s = " ".join(source_text.split())
if norm_q in norm_s:
 return False, "matches only after whitespace normalisation – quote drifted"
head = norm_q[:40]
if head in norm_s:
 return False, f"opening matches but quote diverges after: '{head}...'"
return False, "no match – quotation not present in source"

def main():
 sources = load_sources("data/cases-hero.json")
 outputs = json.load(open("data/model-outputs.json"))

 total = passed = discarded = 0
 print("CITATION VERIFICATION\n" + "=" * 62)

 for case_id, result in outputs["byCase"].items():
 findings = result.get("findings", [])
 print(f"\n{case_id} ({result['demoRole']})")
 if not findings:
 stmt = result.get("noFindingStatement", "")
 expected = ("No supporting evidence identified in the submitted record. "
 "This is not a coverage recommendation.")
 ok = stmt == expected
 print(f" no findings – empty-state wording {'OK' if ok else 'WRONG'}")
 if not ok:
 print(f" expected: {expected}")
 print(f" got: {stmt}")
 continue

 kept = []
 for f in findings:
 total += 1
 ok, reason = verify(f, sources, case_id)
 mark = "PASS" if ok else "DISCARD"
 print(f" [{mark}] {f['documentId']} p{f['page']} – {reason}")
 print(f" \"{f['quote'][:72]}{'...' if len(f['quote']) > 72 else ''}\"")
 if ok:
 passed += 1
 kept.append(f)
 else:
 discarded += 1
 result["verifiedFindings"] = kept

 print("\n" + "=" * 62)
 print(f"{total} quotations checked · {passed} passed · {discarded} discarded")

 if discarded:
 print("\nDiscarded findings never reach a reviewer.")
 return 1
 print("\nAll citations trace to text that provably exists in the source.")
 return 0

if __name__ == "__main__":
 sys.exit(main())

```

## 4 The negative test

A guard that always passes is not a guard. This injects three quotations that must be rejected — one fabricated, one real but attributed to the wrong page, one where the model tidied the wording — plus a genuine quotation as a control that must pass. The safety property is demonstrated rather than asserted.

```
"""
Negative test for the citation guard.

A guard that always passes is not a guard. This injects three fabricated or
drifted quotations and confirms each is discarded, so the safety property is
demonstrated rather than asserted.

Run: python3 scripts/test_guard.py
"""
import json, sys
sys.path.insert(0, "scripts")
from verify_citations import load_sources, verify

sources = load_sources("data/cases-hero.json")
CASE = "PA-2026-114872"

cases = [
 ("fabricated quotation (plausible but never written)", {
 "documentId": "DOC-3", "page": 9,
 "quote": "The patient failed twelve weeks of aggressive conservative therapy including epidural
injections."}),
 ("real sentence, wrong page", {
 "documentId": "DOC-3", "page": 4,
 "quote": "GOALS: 0 of 3 functional goals met."}),
 ("drifted quote – model tidied the wording", {
 "documentId": "DOC-3", "page": 9,
 "quote": "Patient completed a 6 week course of skilled physical therapy without meaningful
improvement."}),
 ("genuine quotation (control – must pass)", {
 "documentId": "DOC-3", "page": 9,
 "quote": "GOALS: 0 of 3 functional goals met."}),
]

print("CITATION GUARD – NEGATIVE TEST\n" + "="*62)
failures = 0
for label, finding in cases:
 ok, reason = verify(finding, sources, CASE)
 should_pass = label.startswith("genuine")
 correct = (ok == should_pass)
 if not correct: failures += 1
 print(f"\n{label}")
 print(f" quote: \"{finding['quote'][:64]}{'...' if len(finding['quote'])>64 else ''}\"")
 print(f" result: {'PASS' if ok else 'DISCARD'} – {reason}")
 print(f" {'correct' if correct else 'GUARD FAILED'}")

print("\n" + "="*62)
print("guard behaved correctly on all 4 probes" if not failures
 else f"{failures} probe(s) mishandled – guard is not sound")
sys.exit(1 if failures else 0)
```

## 5 Verification output

---

Both scripts run against the prototype's own data. Output reproduced exactly.

### Citation verification across all demonstrated cases

```
$ python3 scripts/verify_citations.py

CITATION VERIFICATION
=====

PA-2026-114872 (HERO_1_THE_FIND)
[PASS] DOC-3 p9 – exact match
 "Patient has completed a six-week course of skilled physical therapy with..."
[PASS] DOC-3 p9 – exact match
 "GOALS: 0 of 3 functional goals met."

PA-2026-115003 (HERO_2_THE_OVERRIDE)
[PASS] DOC-2 p5 – exact match
 "Dates of Service: 01/28/2026 through 03/24/2026"

PA-2026-115147 (HERO_3_THE_SILENCE)
no findings – empty-state wording OK

=====
3 quotations checked · 3 passed · 0 discarded

All citations trace to text that provably exists in the source.

$ echo $?
0
```

*The third case returns no findings. That is the correct outcome for it, and the empty-state wording is checked against the exact string the prompt specifies — because the wording of silence is a safety property, not a copy decision.*

## Negative test — fabricated and drifted quotations

```
$ python3 scripts/test_guard.py

CITATION GUARD — NEGATIVE TEST
=====

fabricated quotation (plausible but never written)
quote: "The patient failed twelve weeks of aggressive conservative thera..."
result: DISCARD — no match — quotation not present in source
correct

real sentence, wrong page
quote: "GOALS: 0 of 3 functional goals met."
result: DISCARD — no match — quotation not present in source
correct

drifted quote — model tidied the wording
quote: "Patient completed a 6 week course of skilled physical therapy wi..."
result: DISCARD — no match — quotation not present in source
correct

genuine quotation (control — must pass)
quote: "GOALS: 0 of 3 functional goals met."
result: PASS — exact match
correct

=====
guard behaved correctly on all 4 probes

$ echo $?
0
```

---

**A known limitation, stated rather than hidden.** The prototype's outputs include a case where layer 3 still makes an error the prompt is written to prevent: a complete, well-formatted therapy record for a knee replacement, surfaced in a case about a lumbar complaint. Rule 4 of the prompt does not reliably catch it. That failure is left in the prototype deliberately — it is the reason the deterministic layer runs first, and the reason layer disagreement is surfaced to the reviewer rather than resolved silently.